Node.js Interview Questions

Basic to advanced questions based on the complete_node_js learning projects.

Projects covered: Core Node.js HTTP server, REST API routing, query parameters, static file serving, JSON body parsing, file-system persistence, event emitters, environment variables, and MongoDB/Mongoose connection.

1. Basic Node.js Fundamentals [Beginner]

- What is Node.js, and why is it used for backend development?
- What is the difference between JavaScript running in the browser and JavaScript running in Node.js?
- What are Node.js core modules? Give examples from your project.
- Why did you import modules like node:http, node:path, node:fs/promises, and node:events?
- What is the purpose of package.json in a Node.js project?
- Why did you use "type": "module" in your package file?
- What is the difference between CommonJS and ES Modules?
- What does npm start do in your projects?
- What is the use of node --watch server.js during development?
- What is a callback function? Where do you see callback-style behavior in Node.js?

2. HTTP Server and Request Handling [Beginner to Intermediate]

- How do you create a server using the native http module?
- What are req and res in http.createServer()?
- What happens when the browser sends a request to localhost:8000?
- Why does the server need to call res.end()?
- What is the purpose of server.listen(PORT)?
- How do you set the HTTP status code in Node.js?
- How do you set response headers such as Content-Type?
- What is the difference between sending plain text, JSON, HTML, CSS, and image responses?
- Why should a route return a 404 response when no route matches?
- How would you explain your sendResponse() or sendJSON() helper?

3. Routing and URL Handling [Intermediate]

- How did you manually implement routing without Express?
- What is the difference between req.url and a parsed URL object?
- Why did you create new URL(req.url, baseUrl)?
- What is the difference between pathname and query parameters?
- How does Object.fromEntries(url.searchParams) help with filtering?
- How did you handle routes like /api/country/:country or /api/name/:name?
- Why did you use decodeURIComponent() for route values with spaces?

- What is the difference between path parameters and query parameters?
- When should you use /api/country/India versus /api?country=India?
- How would you add a new route such as /api/continent/:continent?

4. REST API Concepts [Intermediate]

- What is a REST API?
- What is the purpose of your GET /api route?
- What is the purpose of your POST /api route?
- Why should POST usually return status code 201 when data is created?
- What is the difference between 200, 201, 400, 404, and 500?
- How would you validate a request body before saving it?
- How would you prevent duplicate records from being added?
- How would you add PUT, PATCH, and DELETE endpoints to your current API?
- Why is it useful to separate API route logic into handler functions?
- What are some limitations of manually creating REST APIs with only the native http module?

5. Static File Serving [Intermediate]

- What does a static file server do?
- How does your baseDirectory() or serveStatic() utility find files inside the public folder?
- Why do you serve index.html when the request URL is /?
- Why do you need path.join() when working with file paths?
- What is a MIME type, and why is it important?
- How does your getContentType() helper decide whether a file is HTML, CSS, JS, JSON, PNG, or JPG?
- What happens if a requested static file does not exist?
- Why did you create a custom 404.html page?
- What could go wrong if user input is used directly to build file paths?
- How could you improve your static file server for security and caching?

6. File System and JSON Persistence [Intermediate]

- What is the difference between synchronous and asynchronous file-system methods?
- Why did you use fs/promises instead of callback-based fs methods?
- How does your getData() function read and parse JSON data?
- What happens if data.json is empty or contains invalid JSON?
- How does your addNewData() function append a new object to the existing JSON array?
- Why do you need JSON.parse() and JSON.stringify()?
- What are the risks of using a JSON file as a database?
- What could happen if two users submit data at the same time?
- How would you improve file-based persistence for larger data?
- When should you move from JSON file storage to a real database?

7. Request Body Parsing [Intermediate]

- Why does Node.js receive the request body in chunks?
- How does for await (const chunk of req) work?
- Why did you collect chunks into a body string before parsing?
- What does JSON.parse(body) do?
- What error should be returned when the body contains invalid JSON?
- Why should a production API limit the maximum body size?
- How does body parsing become easier when using Express middleware?
- How would you support form data instead of JSON?

8. Event Emitters [Intermediate]

- What is an EventEmitter in Node.js?
- What is the difference between registering a listener with .on() and triggering an event with .emit()?
- In your event emitter example, what happens after the setTimeout() finishes?
- Can one event have multiple listeners? What order do they run in?
- What is a real-world use case for event emitters in backend applications?
- How are event emitters related to asynchronous programming?
- What is the difference between .on() and .once()?
- How would you handle errors emitted by an event emitter?

9. MongoDB and Environment Variables [Intermediate to Advanced]

- Why do backend applications use databases instead of only JSON files?
- What is MongoDB?
- What is Mongoose, and why is it useful?
- Why did you put database connection logic inside a separate config/db.js file?
- What does mongoose.connect(process.env.MONGO_URI) do?
- Why should database URLs be stored in environment variables?
- What does dotenv.config() do?
- Why should .env usually be added to .gitignore?
- Why does your server wait for connectDB() before calling server.listen()?
- What should happen if the database connection fails?

10. Error Handling and Debugging [Advanced]

- Why should asynchronous code be wrapped in try/catch?
- What is the difference between operational errors and programming errors?
- How would you debug a route that always returns 404?
- How would you debug a JSON file that is not saving new data?
- How would you debug a static CSS file that is not loading in the browser?
- Why should internal error messages not always be sent directly to the client?

- How would you add logging to your server?
- How would you test your GET /api and POST /api endpoints?
- What tools can be used to test APIs?
- How would you organize tests for utility functions like getContentType() and filterQueryParams()?

11. Security and Production Thinking [Advanced]

- Why should user input be validated and sanitized?
- What problem does sanitize-html help solve?
- What is cross-site scripting, also known as XSS?
- How could unsafe file paths lead to path traversal attacks?
- How would you prevent access to files outside the public directory?
- Why should APIs avoid accepting unlimited request body sizes?
- What is CORS, and when might your API need it?
- What headers would you add to improve basic security?
- Why is input validation important before writing to a file or database?
- What changes would you make before deploying this app publicly?

12. Architecture and Code Quality [Advanced]

- Why is it better to separate server setup, route handlers, and utility functions?
- How does modular code make debugging easier?
- What is single responsibility principle? Give an example from your utilities.
- How would you refactor repeated response-sending logic?
- How would you structure the project if it grew into a larger backend application?
- When would you choose Express.js instead of only the native http module?
- What middleware would you add in an Express version of this app?
- How would you design a service layer between route handlers and data storage?
- How would you make your API easier for frontend developers to consume?
- How would you document this API for other developers?

13. Project-Based Interview Questions [Tell me about your work]

- Explain your complete_node_js project from start to finish.
- Why did you first build APIs with the native http module instead of directly using Express?
- Explain the request flow for GET /api?country=India.
- Explain the request flow for submitting a new ghost encounter using POST /api.
- How does your app decide whether to serve an API response or a static file?
- What was the most important thing you learned while building route handlers manually?
- What problem did you solve by creating helper functions like sendResponse() and getContentType()?
- What is one bug you faced or could face in this project, and how would you fix it?
- How would you convert your JSON-file backend into a MongoDB-backed backend?

- What are the next features you would add to make this project production-ready?

14. Quick Answer Hints

How to explain your project in an interview

"I built a Node.js backend using the native HTTP module to understand server fundamentals. The project handles REST API routes, query parameters, static file serving, JSON request bodies, file-based persistence, event emitters, and MongoDB connection setup. I separated logic into route handlers and utilities so the code is easier to maintain."

Most important topics to revise

HTTP request-response flow, routing, status codes, MIME types, asynchronous file I/O, JSON parsing, event emitters, environment variables, MongoDB connection flow, and error handling.

Best short project pitch

"This repository shows my Node.js backend learning path: starting with custom HTTP servers and manual routing, then building REST APIs, static file serving, JSON persistence, event-driven code, and MongoDB connection basics."